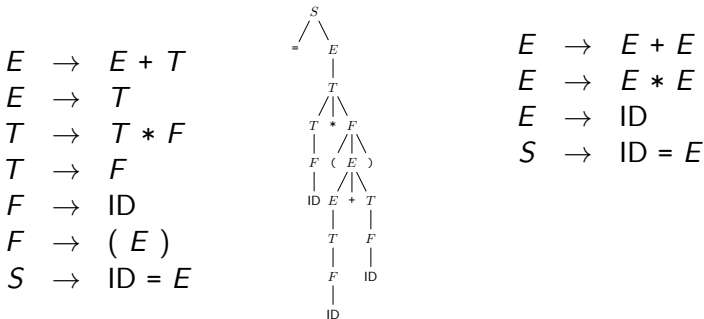


# コンパイラ (7)

2026/10/23

## 抽象構文

ID=ID\*(ID+ID)



具体構文および構文木

抽象構文

抽象構文：具体構文から意味に関わる部分を抽出

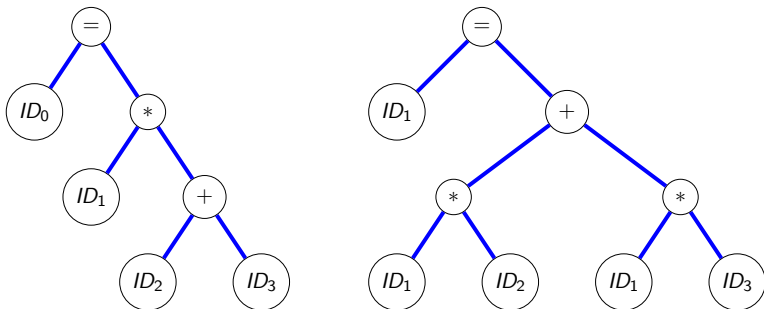
## 抽象構文木

解析木からコード生成に必要な部分を残して変形したもの

内部接点＝演算子、葉＝オペランド

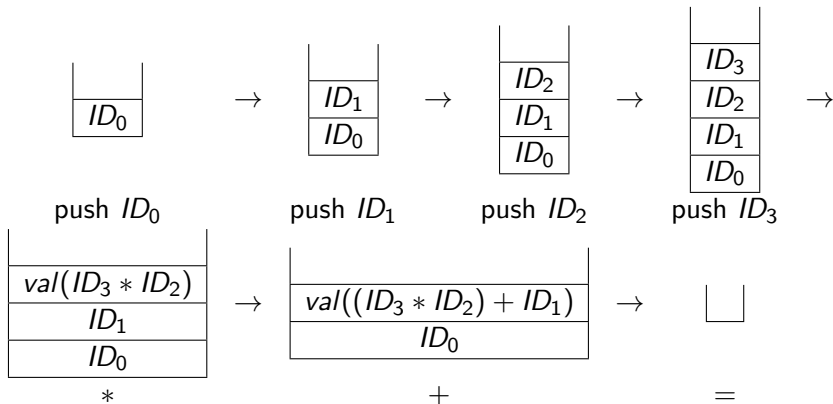
構文主導型変換によって中間コードを生成する際に便利

等価な意味を持つ抽象構文木＝最適化に利用



## 逆ポーランド記法

抽象構文木の後置記法: スタックを用いた実行に対応



### 3 番地コード

$C = A \theta B$      $+, -, *, /, \text{and}, \text{or}, >, >=, =, ! =, <=, <$

$B = \theta A$      $-, \text{not}$

$B = A$

Jump on condition

if  $A$  goto  $L$

if  $\text{not} A$  goto  $L$

Unconditional Jump

goto  $L$

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

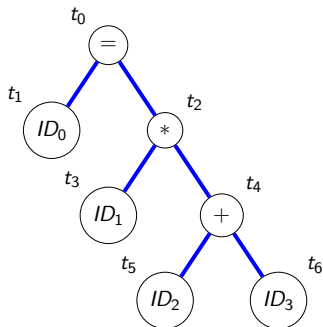
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



抽象構文木に一時変数  $t_i$  を割り当てる

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

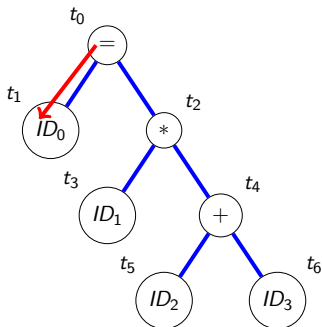
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし， $t_1 = ID_0$  を生成

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

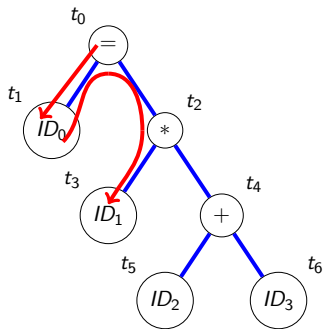
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし， $t_3 = ID_1$  を生成



## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

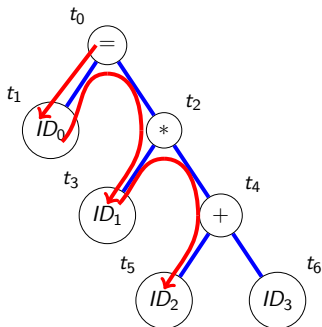
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし， $t_5 = ID_2$  を生成

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

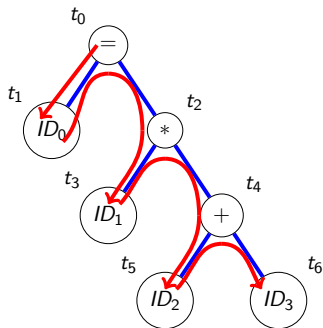
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし,  $t_6 = ID_3$  を生成

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

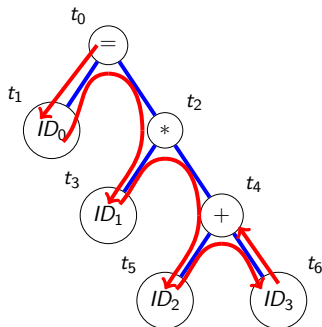
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし、 $t_4 = t_5 + t_6$  を生成

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

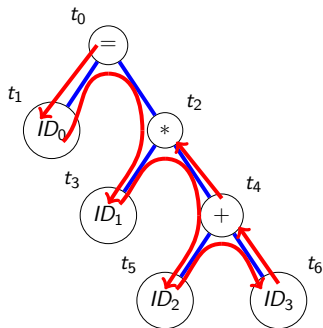
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし， $t_2 = t_3 * t_4$  を生成

## 抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

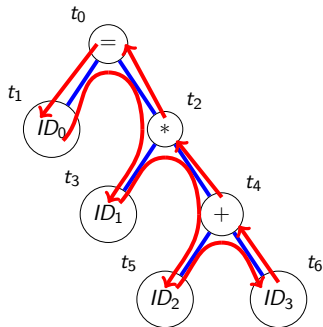
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし、 $t_0 = (t_1 = t_2)$  を生成

## 手続き呼び出しの3番地コード

```
int fib(int x) {
    if (x=0) {
        return 0;
    } else if (x=1) {
        return 1;
    } else {
        return fib(x-2)+fib(x-1);
    }
}

print fib(4);

define @fib(x) {
    t1 := x
    t2 := (x=0)
    if not t1 goto l1
    return 0
l1: t3 := (x=1)
    if not t2 goto l2
    return 1
l2: t4 := t1 - 2
    t5 := call fib(t4)
    t6 := t1 - 1
    t7 := call fib(t6)
    t8 := t5 + t7
    return t8
}

define @main() {
    t1 := 4
    t2 := call fib(t1)
    print t2
}
```

定義: `define @関数名 (仮引数リスト)`

呼出: `call @関数名 (実引数リスト)`

## 実行時環境

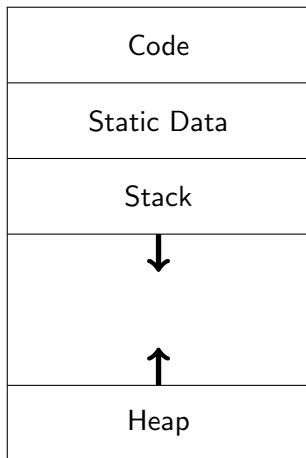
### メモリ領域の割り当て方

- 大域変数＝ヒープ領域 (プロセス内のメモリ領域) にわりあてる。

`malloc, free`

演習のコンパイラでは、スタック領域の底にわりあてる (pop しない)。

- 局所変数＝スタックにわりあてる
- 関数・手続の引数のわりあて (特殊な局所変数)



## 静的割当てと動的割当て

**静的割当て:** 実行の最初から最後までメモリ領域が変化しない  
**動的割り当て:** メモリ領域が実行時に決定  
実行終了時に領域が不要になる

```
int a;  
  
int func(int x) {  
    int y;  
    y = x * x;  
    return (y+y);  
}  
  
main() {  
    int b=1;  
    a=2;  
    printf("%d\n",func(a)+b);  
}
```

静的割当て: a,func,main

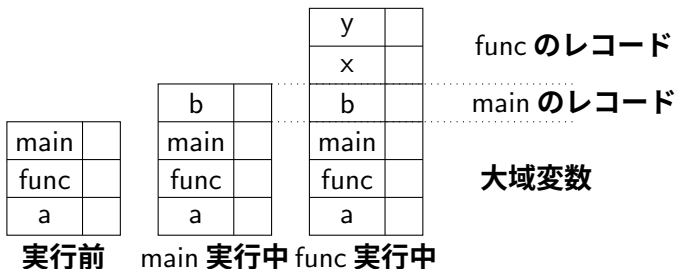
動的割当て: x,y,b



## 駆動レコード

手続き/関数の一回の実行に必要な局所情報

PC(Program Counter)、**返り値、参照可能なシンボルの集合**



## 駆動レコードの構成

|         |
|---------|
| 一時変数領域  |
| 局所データ領域 |
| 静的リンク   |
| 動的リンク   |
| 戻り番地    |
| 実引数計算領域 |
| 戻り値     |
| 退避領域    |

← FP

関数内での計算の一時領域

局所宣言された変数の領域

次に外へ参照すべきレコード

Return 後に復元すべきレコード

Return 後に再開する PC

呼出の際に引数受け渡しに使う

関数の計算結果を格納

呼出の際の一時領域

## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

?  
a=?

## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
→   a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

?  
a=2

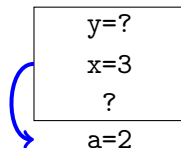
## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
→ printf("func(3)=%d",  
        func(3));  
}
```

?  
a=2

## 変数の参照

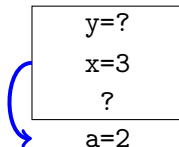
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

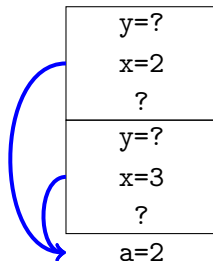
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}  
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

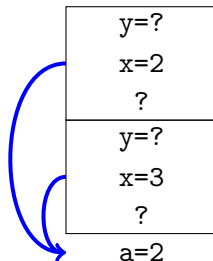


静的リンク



## 変数の参照

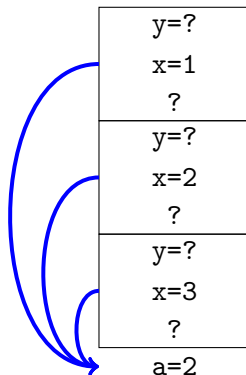
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

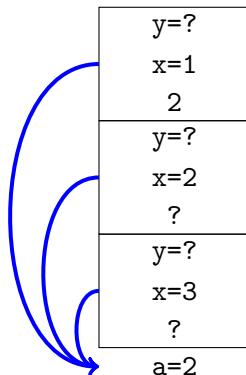
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}  
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

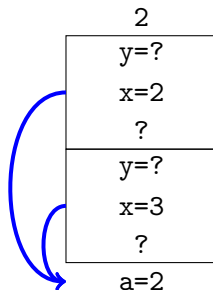
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

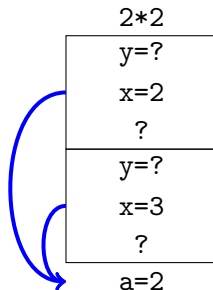
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

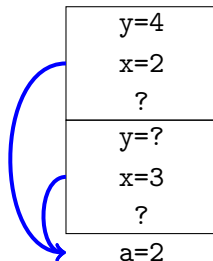
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

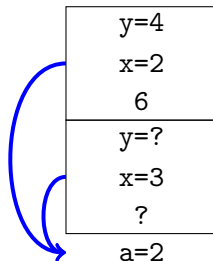
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

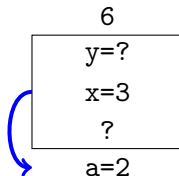
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        → return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

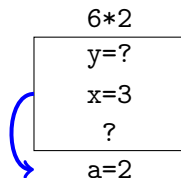


静的リンク



## 変数の参照

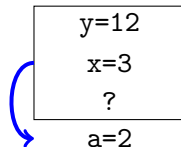
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

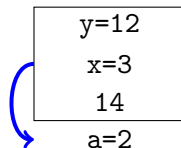
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
→      return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

## 変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
→   a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

14  
a=2

## 変数の参照/記憶位置の決定

局所変数: フレームポインタからの相対位置

大域変数: ヒープ内のアドレス

記号表にはスタックフレームレジスタからの相対番地=何番目に宣言でこういった型をもつか。

型情報=確保する領域量 (実習の場合は整数のみ)

関数の引数うけわたりは局所変数の宣言+代入

## 動作意味

$$\langle \{S, E, T, F\}, \{+, *, (, ), n\}, P, S \rangle$$

$$\left\{ \begin{array}{ll} S \rightarrow E\#, & T \rightarrow F, \\ E \rightarrow E + T, & F \rightarrow ( E ), \\ E \rightarrow T, & F \rightarrow n, \\ T \rightarrow T * F \end{array} \right\}$$

各生成規則は値を計算する意味を持つ。

$E \rightarrow E + T$ : 右辺の  $E$  の値と  $T$  の値を加算した値を左辺の  $E$  の値とする。

$\Rightarrow$  '+' に加算の意味を与える。

## 構文に従った動作意味

$$\begin{array}{ll} S \rightarrow E\# & \text{var}(S) = \text{var}(E) \\ E \rightarrow E_1 + T & \text{var}(E) = \text{var}(E_1) + \text{var}(T) \\ E \rightarrow T & \text{var}(E) = \text{var}(T) \\ T \rightarrow T_1 * F & \text{var}(T) = \text{var}(T_1) \times \text{var}(F) \\ T \rightarrow F & \text{var}(T) = \text{var}(T) \\ F \rightarrow ( E ) & \text{var}(F) = \text{var}(E) \\ F \rightarrow i & \text{temp} = \text{val}(i); \text{var}(F) = \text{temp} \end{array}$$

### 構文主導型変換

**インタプリタ：** 生成規則に対応する動作をする。

**コンパイラ：** 生成規則に対応した目的コード（中間コード）を生成する。

## 構文主導型変換

CF 文法の各生成規則に対してコード生成ルーチンをわりあてる。

- ▶ newTemp()  
変数  $T$  の領域を確保して、記号番地または一時変数へのポインタを返す。
- ▶ newLabel()  
新たなラベル名を返す。
- ▶ generate(中間コード)  
中間コードを出力ファイルに出力する。

$n$  回目に newTemp() を呼び出したとき、生成される変数  $t_n$

$m$  回目に newLabel() を呼び出したとき、生成されるラベル  $L_m$

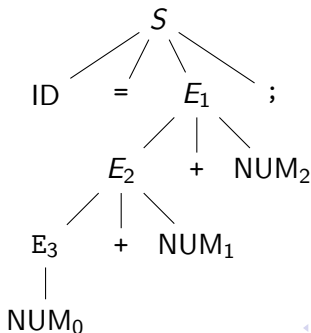


## 構文主導型変換の基礎

$$S \rightarrow ID = E ;$$

$$E \rightarrow E^{(1)} + NUM$$

$$E \rightarrow NUM$$

$$ID = NUM_0 + NUM_1 + NUM_2 ;$$


## 中間コード (3 番地コード) 生成

| 生成規則                                 | 出力される 3 番地コード                                                                               |
|--------------------------------------|---------------------------------------------------------------------------------------------|
| $E_3 \rightarrow \text{NUM}_0$       | $t3 = \text{NUM}_0.val$                                                                     |
| $E_2 \rightarrow E_3 + \text{NUM}_1$ | $t2 = t3 + \text{NUM}_1.val$                                                                |
| $E_1 \rightarrow E_2 + \text{NUM}_2$ | $t1 = t2 + \text{NUM}_2.val$                                                                |
| $S \rightarrow \text{ID} = E_1 ;$    | $\text{ID.name} = t1$                                                                       |
| $E \rightarrow \text{NUM}$           | $\{E.var := \text{newTemp}();$<br>$\text{generate}('E.var = \text{NUM.val}')$               |
| $E \rightarrow E^{(1)} + \text{NUM}$ | $\{E.var := \text{newTemp}();$<br>$\text{generate}('E.var = E^{(1)}.var + \text{NUM.val}')$ |
| $S \rightarrow \text{ID} = E ;$      | $\{\text{generate}('ID.name = E.var')$                                                      |

## IF 文の中間コード生成

文  $\rightarrow$  if '(' 式 ')' 文<sup>(1)</sup>

1. ' 式 ' のコードを生成 t1 に結果収納
2. if not t1 goto l1
3. ' 文 '<sup>(1)</sup> のコードを生成
4. l1:

## IF 文の中間コード生成

文  $\rightarrow$  if '(' 式 ')' 文<sup>(1)</sup> else 文<sup>(2)</sup>

1. ' 式' のコードを生成 t1 に結果収納
2. if not t1 goto l1
3. ' 文'<sup>(1)</sup> のコードを生成
4. goto l2
5. l1:
6. ' 文'<sup>(2)</sup> のコードを生成
7. l2:

## while 文の中間コード生成

文  $\rightarrow$  while '(' 式 ')' 文<sup>(1)</sup>

1. l1:
2. ' 式' のコードを生成 t1 に結果収納
3. if not t1 goto l2
4. ' 文'<sup>(1)</sup> のコードを生成
5. goto l1
6. l2:

## 構文主導型変換によるコード生成

```
文 → if ( { 文.label:=newLabel();}  
式      (式のコード)  
)      {generate('if_not 式.var goto 文.label')}  
文(1)  (文(1) のコード){generate('label 文.label');  
      文(1).label:=newLabel();generate('goto 文(1).label')}  
else    {generate('label 文.label')}  
文(2)    {generate('label 文(1).label')}
```

```
文 → while ( { 文.label1:=newLabel();  
              generate('label 文.label1')}  
式      (式のコード)  
)      { 文.label2:=newLabel();  
          generate('if_not 式.var goto 文.label2')}  
文(1)  (文のコード)  
      {generate('goto 文.label1');  
        generate('label 文.label2')}
```

## PDTを用いた構文主導型変換

## コード生成を行うタイミング＝還元操作

```

1:  $E \rightarrow E^{(1)} + A$      $\{E.var := \text{newLabel}();$   

                            $\text{generate}('E.var := E^{(1)}.var + A.var')\}$   

2:  $E \rightarrow A$               $\{E.var := \text{newTemp}();$   

                            $\text{generate}('E.var := A.var')\}$   

3:  $A \rightarrow A^{(1)} * \text{NUM}$    $\{A.var := \text{newTemp}();$   

                            $\text{generate}('A.var := A^{(1)}.var \times \text{NUM}.val')\}$   

4:  $A \rightarrow \text{NUM}$             $\{A.var := \text{newTemp}();$   

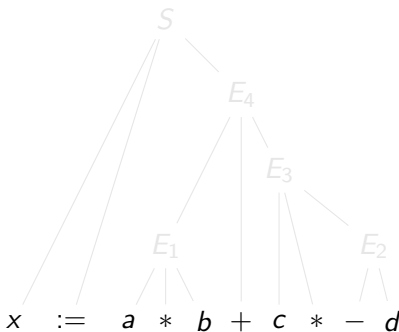
                            $\text{generate}('A.var := \text{NUM}.val')\}$ 

```

|                                  |        |                        |
|----------------------------------|--------|------------------------|
| $\vdash S_0$                     | 3+4*5# |                        |
| $\vdash S_0nS_6$                 | +4*5#  | s6                     |
| $\vdash S_0TS_7$                 | +4*5#  | t1=3<br>r4             |
| $\vdash S_0ES_1$                 | +4*5#  | t2=3<br>r2             |
| $\vdash S_0ES_1+S_2$             | 4*5#   | s2                     |
| $\vdash S_0ES_1+S_2nS_6$         | *5#    | s6                     |
| $\vdash S_0ES_1+S_2TS_3$         | *5#    | t3=5<br>r4             |
| $\vdash S_0ES_1+S_2TS_3*S_4$     | 5#     | s4                     |
| $\vdash S_0ES_1+S_2TS_3*S_4nS_5$ | #      | s5                     |
| $\vdash S_0ES_1+S_2TS_3$         | #      | t4=t3 $\times$ 5<br>r3 |
| $\vdash S_0ES_1$                 | #      | t5=t2+t4<br>r1         |
| $\vdash acc$                     |        |                        |

## 変換例

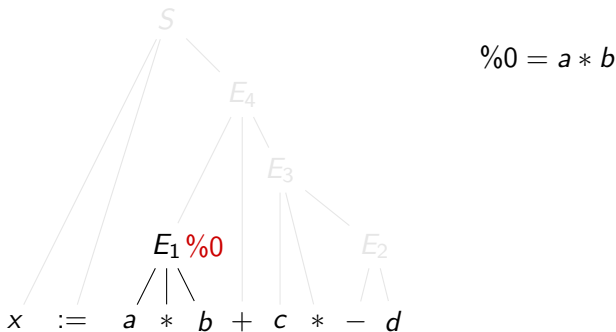
$$\begin{aligned}
 E_1 &\rightarrow a * b & \{\%0 = a * b\} \\
 E_2 &\rightarrow -d & \{\%1 = -d\} \\
 E_3 &\rightarrow c * E_2 & \{\%2 = c * \%1\} \\
 E_4 &\rightarrow E_1 + E_2 & \{\%3 = \%0 + \%2\} \\
 S &\rightarrow x := E_3 & \{x = \%3\}
 \end{aligned}$$





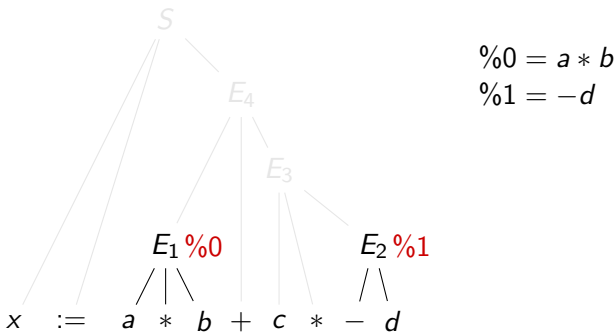
## 変換例

$$\begin{aligned}
 E_1 &\rightarrow a * b & \{\%0 = a * b\} \\
 E_2 &\rightarrow -d & \{\%1 = -d\} \\
 E_3 &\rightarrow c * E_2 & \{\%2 = c * \%1\} \\
 E_4 &\rightarrow E_1 + E_2 & \{\%3 = \%0 + \%2\} \\
 S &\rightarrow x := E_3 & \{x = \%3\}
 \end{aligned}$$

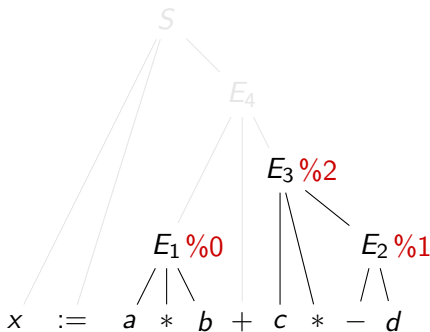


$\%0 = a * b$

## 变换例

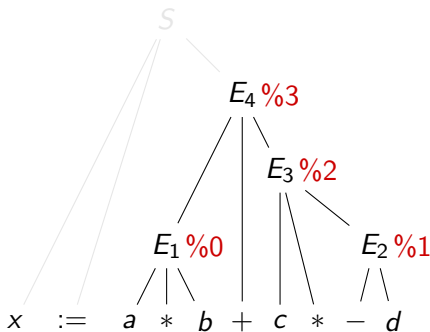
$$E_1 \rightarrow a * b \quad \{ \%0 = a * b \}$$
$$E_2 \rightarrow -d \quad \{\%1 = -d\}$$
$$E_3 \rightarrow c * E_2 \quad \{\%2 = c * \%1\}$$
$$E_4 \rightarrow E_1 + E_2 \quad \{\%3 = \%0 + \$2\}$$
$$S \rightarrow x := E_3 \quad \{x = \%3\}$$


## 变换例

$$\begin{array}{ll} E_1 \rightarrow a * b & \{\%0 = a * b\} \\ E_2 \rightarrow -d & \{\%1 = -d\} \\ E_3 \rightarrow c * E_2 & \{\%2 = c * \%1\} \\ E_4 \rightarrow E_1 + E_2 & \{\%3 = \%0 + \%2\} \\ S \rightarrow x := E_3 & \{x = \%3\} \end{array}$$

$$\begin{aligned}\%0 &= a * b \\ \%1 &= -d \\ \%2 &= c * \%1\end{aligned}$$

## 変換例

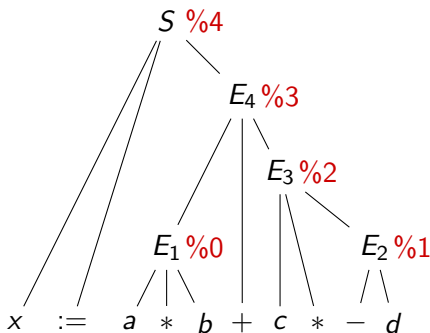
$$\begin{aligned}
 E_1 &\rightarrow a * b & \{\%0 = a * b\} \\
 E_2 &\rightarrow -d & \{\%1 = -d\} \\
 E_3 &\rightarrow c * E_2 & \{\%2 = c * \%1\} \\
 E_4 &\rightarrow E_1 + E_2 & \{\%3 = \%0 + \%2\} \\
 S &\rightarrow x := E_3 & \{x = \%3\}
 \end{aligned}$$



$\%0 = a * b$   
 $\%1 = -d$   
 $\%2 = c * \%1$   
 $\%3 = \%0 + \%2$

## 変換例

$$\begin{array}{ll} E_1 \rightarrow a * b & \{\%0 = a * b\} \\ E_2 \rightarrow -d & \{\%1 = -d\} \\ E_3 \rightarrow c * E_2 & \{\%2 = c * \%1\} \\ E_4 \rightarrow E_1 + E_2 & \{\%3 = \%0 + \%2\} \\ S \rightarrow x := E_3 & \{x = \%3\} \end{array}$$

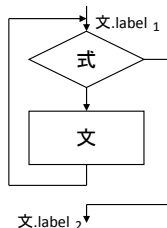


$\%0 = a * b$   
 $\%1 = -d$   
 $\%2 = c * \%1$   
 $\%3 = \%0 + \%2$   
 $x = \%3$

## 条件分岐に対する構文主導型変換

分岐条件の評価の後に分岐命令を生成する必要がある。

```
文 → WHILE '(', { 文.label1 := newLabel();  
                  generate('goto 文.label1');  
                  式 ')', { 文.label2 := newLabel();  
                           generate('if_not 式.var goto 文.label2 ');  
                  文(1) {generate('goto 文.label1');  
                       generate('label 文.label2');}
```



## while 文の中間コード生成例

```
while(x<20) {  
    r=r+x;  
    x=x+1  
}
```

```
L1 := newLabel()  
generate('labelL1')  
generate('t1:=x < 20')  
generate('labelL2')  
generate('if_not t1 goto L2')  
generate('t2:=r + x')  
generate('r:=t2')  
generate('t3:=x + 1')  
generate('x:=t3')  
generate('gotoL1')  
generate('labelL2')
```

|     |                   |
|-----|-------------------|
| L1: | t1:=x<20          |
|     | if not t1 goto L2 |
|     | t2:=r+x           |
|     | r:=t2             |
|     | t3:=x+1           |
|     | x:=t3             |
|     | goto L1)          |
| L2: |                   |

## 上昇型構文解析におけるコード生成

### IF 文

```
I  →  if ( E )      { S.label1:=newLabel();  
                      generate('if_not E.var goto S.label1') }  
S'  →  IS(1)        { S'.label := newLabel();  
                      generate('goto S'.label');  
                      generate('label I.label') }  
S   →  S' else S(2)  { generate('label S'.label') }
```

### WHILE 文

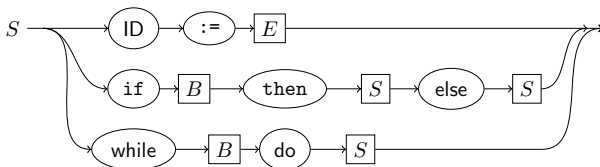
```
I  →  while (      { I.label:=newLabel();  
                      generate('label I.label') }  
S'  →  E )          { S'.label := newLabel();  
                      generate('if_not E.var goto S'.label') }  
S   →  IS'S(1)      { generate('goto I.label');  
                      generate('label S'.label') }
```



## 下降型構文主導型変換

構文図式に従ってコードを生成する。

**丸**の部分でコードを生成する



```
// nextsym には先読みトークンがあらかじめ格納されている
If nextsym.kind = ID then // 代入文
    getNextsym()
    If nextsym.kind = '=' then // 代入記号
        E のコードを生成 ($tempE に結果を格納) // E
        generate('var(ID.val) = $tempE') // 代入本体
    else error()
else if nextsym.kind = if then // 条件文
    getNextsym()
    B のコードを生成 ($tempB に結果を格納) // B
    L1 = newlabel() // 偽の場合の飛び先
    generate('if_not $tempB goto L1') // 分岐命令本体
    S のコードを生成 // 条件が真の場合の S
    L2 = newlabel() // 偽の命令の後への飛び先
    generate('goto L2')
    generate('label L1')
    S のコードを生成 // 条件が偽の場合の S
    generate('label L2') // 真の S のジャンプ先
else if nextsym.kind = while then // 繰り返し文
    getNextsym()
    L1 = newlabel()
    generate('label L1') // 繰り返しで戻るラベル
    B のコードを生成 ($tempB に結果を格納) // B
    If nextSym.kind = do then
        L2 = newlabel()
        generate('if_not $tempB goto L2')
        S のコードを生成 // 繰り返し本体の S
        generate('goto L1') // 繰り返しに戻る
        generate('label L2') // 脱出先のラベル
    else error()
else
    error()
```

## 属性文法

属性文法 (attribute grammar): 抽象構文木に属性を加えた文法  
構文主導型変換への応用=コード生成に関する情報を属性として扱う

属性文法  $A_g = (G, \mathcal{A}, E)$

$G = (N, \Sigma, P, S)$  は CF 文法

$\mathcal{A}$ : 属性の集合

$\mathcal{A} = \cup_{X \in N \cup \Sigma} \mathcal{A}(X)$ .  $a \in \mathcal{A}(X)$  のとき、 $X.a$  と書く。

$E$ : 意味規則の集合

$p \in P$  に対して以下のような規則定める。

$$p\{X_i.a_i = f_i(X_{i_1}.a_{i_1}, \dots, X_{i_n}.a_{i_n})\}$$

$$E \rightarrow E^{(1)} + E^{(2)} \quad \{E.val = E^{(1)}.val + E^{(2)}.val\}$$

$$E \rightarrow \text{NUM} \quad \{E.val = \text{NUM}.val\}$$

## 属性の種類

合成 (synthesized) 属性

右辺の属性から合成された左辺の属性:

相続 (inherited) 属性 (「継承属性」とも言う)

合成以外の属性

$$G = (\{T, L\}, \{\text{MAX}, \text{MIN}, \text{NUM}, '(', ')', ', ', P, T\})$$

$\mathcal{A}$ :

|     | 合成属性    | 相続属性   |
|-----|---------|--------|
| $T$ | $T.val$ |        |
| $L$ | $L.val$ | $L.op$ |

$$P = \left\{ \begin{array}{ll} T \rightarrow \text{MIN} ' ( ' L ' ) ' & \{L.op = \text{MIN}, T.val = L.val\} \\ T \rightarrow \text{MAX} ' ( ' L ' ) ' & \{L.op = \text{MAX}, T.val = L.val\} \\ L \rightarrow \text{NUM} & \{L.val = \text{NUM}.val\} \\ L \rightarrow L^{(1)} ' , ' \text{ NUM} & \left\{ \begin{array}{l} L.val = f(L.op, L^{(1)}.val, \text{NUM}.val), \\ L^{(1)}.op = L.op \end{array} \right\} \end{array} \right\}$$

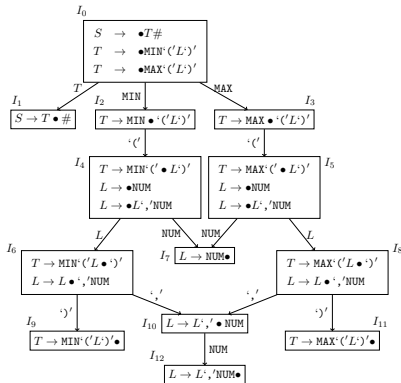
## 属性文法に基づくコード生成例

$G$  に生成規則  $\{S \rightarrow T\#\}$  を加えた拡張文法に対する LR 解析

$$P = \left\{ \begin{array}{ll} 0: S \rightarrow T\#, & 3: L \rightarrow \text{NUM}, \\ 1: T \rightarrow \text{MIN } ' ( ' L ' ) ', & 4: L \rightarrow L ' , ' \text{NUM}, \\ 2: T \rightarrow \text{MAX } ' ( ' L ' ) ' & \end{array} \right\}$$

$$\text{Follow}_1(T) = \{\#\}, \text{Follow}_1(L) = \{), ', '\}$$

## 属性文法に基づくコード生成例



|          | action |     |     |     |     |    |     | GOTO |   |
|----------|--------|-----|-----|-----|-----|----|-----|------|---|
|          | MIN    | MAX | NUM | (   | )   | ,  | #   | T    | L |
| $I_0$    | s2     | s2  |     |     |     |    |     | 1    |   |
| $I_1$    |        |     |     |     |     |    | acc |      |   |
| $I_2$    |        |     | s4  |     |     |    |     |      |   |
| $I_3$    |        |     | s4  |     |     |    |     |      |   |
| $I_4$    |        |     | s7  |     |     |    |     |      | 6 |
| $I_5$    |        |     | s7  |     |     |    |     |      | 8 |
| $I_6$    |        |     |     | s9  | s10 |    |     |      |   |
| $I_7$    |        |     |     | r3  | r3  |    |     |      |   |
| $I_8$    |        |     |     | s11 | s10 |    |     |      |   |
| $I_9$    |        |     |     |     |     | r1 |     |      |   |
| $I_{10}$ |        |     | s12 |     |     |    |     |      |   |
| $I_{11}$ |        |     |     |     |     | r2 |     |      |   |
| $I_{12}$ |        |     |     | r4  | r4  |    |     |      |   |

## SLR(1) 構文解析表

## LR(0) 遷移系

## 属性文法に基づくコード生成例

|   |                                            |                        |             |                                                         |
|---|--------------------------------------------|------------------------|-------------|---------------------------------------------------------|
|   | $(l_0,$                                    | $\text{MIN}(3,5,7)\#,$ | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MIN}l_2,$                       | $(3,5,7)\#,$           | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_4,$                   | $3,5,7)\#,$            | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_43l_7,$               | $,5,7)\#,$             | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,$               | $,5,7)\#,$             | $3)$        | $\%1.val = 3$                                           |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10},$        | $5,7)\#,$              | $3)$        |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10}5l_{12},$ | $,7)\#,$               | $3)$        |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,$               | $,7)\#,$               | $34)$       | $\%2.val = f(\%1.op, \%1.val, 5);$<br>$\%2.op = \%1.op$ |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10},$        | $7)\#,$                | $34)$       |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10}7l_{12},$ | $)\#,$                 | $34)$       |                                                         |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,$               | $)\#,$                 | $344)$      | $\%3.val = f(\%2.op, \%2.val, 7);$<br>$\%3.op = \%2.op$ |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6)l_9,$           | $\#,$                  | $344)$      |                                                         |
| ⊢ | $(l_0Tl_1,$                                | $\#,$                  | $3441)$     | $\%3.op = \text{min}; \%4.val = \%3.val$                |
| ⊢ | $(acc,$                                    | $\#,$                  | $34410)$    |                                                         |

相続属性 ( $f$  の第一引数) が最後に決定される

$T.val = \%4.val$

## 属性文法に基づくコード生成例

|   |                                            |                        |                |                                                      |
|---|--------------------------------------------|------------------------|----------------|------------------------------------------------------|
|   | $(l_0,$                                    | $\text{MIN}(3,5,7)\#,$ | $\varepsilon)$ |                                                      |
| ⊢ | $(l_0\text{MIN}l_2,$                       | $(3,5,7)\#,$           | $\varepsilon)$ |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_4,$                   | $3,5,7)\#,$            | $\varepsilon)$ |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_43l_7,$               | $,5,7)\#,$             | $\varepsilon)$ |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,$               | $,5,7)\#,$             | $3)$           | $\%1.\text{val} = 3$                                 |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10},$        | $5,7)\#,$              | $3)$           |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10}5l_{12},$ | $,7)\#,$               | $3)$           |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,$               | $,7)\#,$               | $34)$          | $\%2.\text{val} = f(\text{min}, \%1.\text{val}, 5);$ |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10},$        | $7)\#,$                | $34)$          |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,l_{10}7l_{12},$ | $)\#,$                 | $34)$          |                                                      |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6,$               | $)\#,$                 | $344)$         | $\%3.\text{val} = f(\text{min}, \%2.\text{val}, 7);$ |
| ⊢ | $(l_0\text{MIN}l_2(l_4Ll_6)l_9,$           | $\#,$                  | $344)$         |                                                      |
| ⊢ | $(l_0Tl_1,$                                | $\#,$                  | $3441)$        | $\%4.\text{val} = \%3.\text{val}$                    |
| ⊢ | $(acc,$                                    | $\#,$                  | $34410)$       |                                                      |

相続属性 ( $f$  の第一引数) をバックパッチによって関数を書き換える

$T.\text{val} = \%4.\text{val}$



## 属性文法に基づくコード生成例

|   |                                             |                          |             |                                                         |
|---|---------------------------------------------|--------------------------|-------------|---------------------------------------------------------|
|   | $(l_0,$                                     | $\text{MAX}(3, 5, 7)\#,$ | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MAX}l_2,$                        | $(3, 5, 7)\#,$           | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_4,$                    | $3, 5, 7)\#,$            | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_43l_7,$                | $, 5, 7)\#,$             | $\epsilon)$ |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6,$                | $, 5, 7)\#,$             | $3)$        | $\%1.val = 3$                                           |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6, l_{10},$        | $5, 7)\#,$               | $3)$        |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6, l_{10}5l_{12},$ | $, 7)\#,$                | $3)$        |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6,$                | $, 7)\#,$                | $34)$       | $\%2.val = f(\%1.op, \%1.val, 5);$<br>$\%1.op = \%2.op$ |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6, l_{10},$        | $7)\#,$                  | $34)$       |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6, l_{10}7l_{12},$ | $)\#,$                   | $34)$       |                                                         |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6,$                | $)\#,$                   | $344)$      | $\%3.val = f(\%2.op, \%2.val, 7);$<br>$\%2.op = \%3.op$ |
| ⊢ | $(l_0\text{MAX}l_2(l_4Ll_6)l_9,$            | $\#,$                    | $344)$      |                                                         |
| ⊢ | $(l_0Tl_1,$                                 | $\#,$                    | $3442)$     | $\%3.op = \text{max}; \%4.val = \%3.val$                |
| ⊢ | $(acc,$                                     | $\#,$                    | $34410)$    |                                                         |

相続属性 ( $f$  の第一引数) が最後に決定される

$T.val = \%4.val$

## 属性文法に基づくコード生成例

|   |                                                         |              |                 |                             |
|---|---------------------------------------------------------|--------------|-----------------|-----------------------------|
|   | ( $l_0$ ,                                               | MAX(3,5,7)#, | $\varepsilon$ ) |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ,                                     | (3,5,7)#,    | $\varepsilon$ ) |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ ,                             | 3,5,7)#,     | $\varepsilon$ ) |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ 3 $l_7$ ,                     | ,5,7)#,      | $\varepsilon$ ) |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ ,                       | ,5,7)#,      | 3)              | %1.val = 3                  |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ , $l_{10}$ ,            | 5,7)#,       | 3)              |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ , $l_{10}$ 5 $l_{12}$ , | ,7)#,        | 3)              |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ ,                       | ,7)#,        | 34)             | %2.val = f(max, %1.val, 5); |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ , $l_{10}$ ,            | 7)#,         | 34)             |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ , $l_{10}$ 7 $l_{12}$ , | )#,          | 34)             |                             |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ ,                       | )#,          | 344)            | %3.val = f(max, %2.val, 7); |
| ⊢ | ( $l_0$ MAX $l_2$ ( $l_4$ $l_6$ ) $l_9$ ,               | #,           | 344)            |                             |
| ⊢ | ( $l_0$ $Tl_1$ ,                                        | #,           | 3442)           | %4.val = %3.val             |
| ⊢ | (acc,                                                   | #,           | 34410)          |                             |

相続属性 ( $f$  の第一引数) をバックパッチによって関数を書き換える

$T.val = \%4.val$

## LLVM による条件文コード

if  $B$  then  $S_1$  else  $S_2$

$\%1 = [B \text{ を計算するコード}]$

br i1  $\%1$ , label  $\%L1$ , label  $\%L2$

$L1:$

$[S_1 \text{ のコード}]$

br label  $\%L3$

$L2:$

$[S_2 \text{ のコード}]$

br label  $\%L3$

$L3:$

$L1, L2, L3$  は実際に定義される前に参照される。  
 $S_2$  の後の  $\%3$  へのジャンプは不要。

## 条件文における論理式

条件文における条件の評価：結果待ち / 短絡型

条件が副作用を持つ場合に違いが出る

```
if (x>100 && (z=y)<30) { 文 1 } else { 文 2 }
```

$z=y$  の評価結果は  $y$  の値

$z=y$  が実行されるか？

結果待ち＝評価される、短絡型＝ $x>100$  のとき評価される

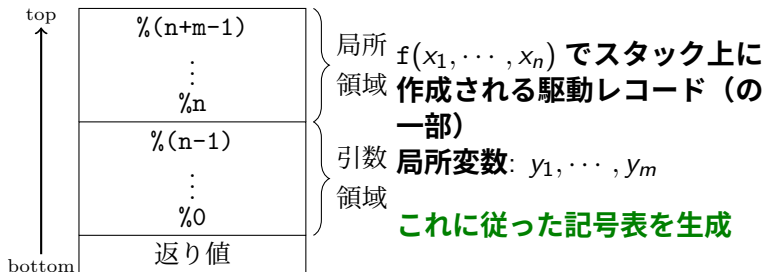
短絡型： 条件文の評価中に if 文を評価する。

結果待： 条件文の式を評価してから if を評価する。

短絡型の評価を行う場合が多い (C, python)

java の場合：&&, || が短絡型, &, | が結果待ち

## 関数引数・返り値と駆動レコード



**値呼び:**  $x = f(e_1, \dots, e_n)$

$e_i$  を評価してその値を  $x_i$  の領域にコピー

**参照呼び:**  $x = f(z_1, \dots, z_n)$

$z_i$  のアドレスを  $x_i$  にコピー

# 引数受渡し

引数=パラメータ (parameter)

## 2 種類の受渡し方法

値渡し：実引数の値を評価して仮引数に代入

参照渡し：実引数の値を評価してポインタを仮引数に代入

### 値呼び

```
void swap(x,y) {  
    int t;  
    t = x;  
    x = y;  
    y = t;  
}
```

### 参照呼び

```
void swap(int *x,int *y) {  
    int t;  
    t = *x;  
    *x = *y;  
    *y = t;  
}
```

値呼びの場合：全く効果なし。

入れ替えられた値は駆動レコード内で実行後破棄

参照呼びの場合：値が入れ替えられる。

swap 中の  $y=x$ ,  $x=t$  の代入先は、実引数の領域

## 値呼びのコンパイル

%0: x の値, %1: y の値が格納されているアドレス

```
void swap(x,y) {  
    int t;  
    t = x;  
    x = y;  
    y = t;  
}  
  
define void @swap(i32, i32) #0 {  
    %3 = alloca i32, align 4  
    %4 = alloca i32, align 4  
    %5 = alloca i32, align 4  
    store i32 %0, i32* %3, align 4  
    store i32 %1, i32* %4, align 4  
    %6 = load i32, i32* %3, align 4  
    store i32 %6, i32* %5, align 4  
    %7 = load i32, i32* %4, align 4  
    store i32 %7, i32* %3, align 4  
    %8 = load i32, i32* %5, align 4  
    store i32 %8, i32* %4, align 4  
    ret void  
}
```

## 参照呼びのコンパイル

%0: x の値, %1: y の値が格納されているアドレス

```
void swap(int *x,int *y) {  
    int t;  
    t = *x;  
    *x = *y;  
    *y = t;  
}
```

```
define void @swap(i32*, i32*) #0 {  
    %3 = alloca i32*, align 8  
    %4 = alloca i32*, align 8  
    %5 = alloca i32, align 4  
    store i32* %0, i32** %3, align 8  
    store i32* %1, i32** %4, align 8  
    %6 = load i32*, i32** %3, align 8  
    %7 = load i32, i32* %6, align 4  
    store i32 %7, i32* %5, align 4  
    %8 = load i32*, i32** %4, align 8  
    %9 = load i32, i32* %8, align 4  
    %10 = load i32*, i32** %3, align 8  
    store i32 %9, i32* %10, align 4  
    %11 = load i32, i32* %5, align 4  
    %12 = load i32*, i32** %4, align 8  
    store i32 %11, i32* %12, align 4  
    ret void  
}
```



## 関数定義/値呼び

```
int add(int x,int y) {  
    return x+y;  
}
```

```
define i32 @add(i32,i32) {  
entry:  
    % x = alloca i32  
    % y = alloca i32  
    store i32 % 0, i32* % x  
    store i32 % 1, i32* % y  
    % 2 = load i32, i32* % x  
    % 3 = load i32, i32* % y  
    % 4 = add i32 % 2, % 3  
    ret i32 % 4  
}
```

関数定義：

define  $\langle$  型  $\rangle$   $\langle$  関数名  $\rangle$  (引数型, ..., 引数型)

$i$  番目の引数  $\%(i-1)$  で参照

関数呼出 (値呼び) :  $\%ret = call\ i32\ @add(\%2,\%3),\ align\ 4$